

---

title: 病种分析课件\_模块三 type: 方法论 tags: [病种分析, 可视化, 数据分析, 机器学习, DRG, DIP]

---

# 模块三：利器与洞察 —— 分析工具与可视化实战

本模块旨在为医院运营管理人员、数据分析师及临床科室管理者提供一整套深度、严谨且具备高度实操性的数据分析与可视化工具箱。从基础的异常值识别，到多维度的深度比对，再到战略层面的宏观定位监控，最后深入到基于机器学习的个体归因预测。我们将通过理论阐述、统计学原理剖析及 Python/R 实战代码，全面赋能病种分析（DRG/DIP）工作，将冰冷的数据转化为管理决策的敏锐洞察力。

---

## 3.1 异常识别与基线确立：从统计描述到极端案例甄别

在 DRG/DIP 支付制度下，费用超支的“离群值”（Outliers）往往是吞噬科室利润的黑洞。精准识别这些异常病例，不仅是成本控制的第一步，也是制定临床路径（Clinical Pathway）基线的必要前提。

### 3.1.1 统计学理论：描述性统计与 IQR 准则

**描述性统计（Descriptive Statistics）：** 在对任何病种数据进行复杂建模前，首先需要了解数据的整体分布面貌。主要包括：

- **集中趋势度量：** 均值（Mean）、中位数（Median，对偏态分布更具代表性）。在医疗费用数据中，通常呈明显的右偏（正偏态）分布，即少数极高费用拉高了整体均值，此时中位数更能反映“一般患者”的费用水平。
- **离散程度度量：** 标准差（Standard Deviation）、变异系数（Coefficient of Variation,  $CV = \text{Standard Deviation} / \text{Mean}$ ）。CV 越小，说明该病种内病例间的费用差异越小，临床同质性越高；在 DRG 分组评价中，组内 CV 是一项关键指标。

**四分位距（Interquartile Range, IQR）法则：** Boxplot（箱线图）是识别离群值的经典可视化工具。它基于数据的五数概括（最小值、下四分位数 Q1、中位数 Q2、上四分位数 Q3、最大值）。

- $IQR = Q3 - Q1$
- 异常值下界：  $Q1 - 1.5 \times IQR$
- 异常值上界：  $Q3 + 1.5 \times IQR$

在医疗费用分析中，超过上界的病例通常被称为高倍率病例，需重点进行病案首页质控和临床合理用药/耗材核查。

### 3.1.2 Python 实战：DRG 组内极端费用病例可视化

下面是一段使用 `pandas`、`seaborn` 和 `matplotlib` 构建的 Python 代码，它生成了虚拟数据并绘制了包含抖动散点（Stripplot）的箱线图，以便直观地定位某 DRG 组内的极端高费病例。

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# 1. 构造虚拟 DRG 费用数据
np.random.seed(42)
n_patients = 500
# 绝大部分患者费用服从对数正态分布（模拟右偏的医疗费用）
base_costs = np.random.lognormal(mean=9.5, sigma=0.4, size=n_patients)
drg_groups = np.random.choice(['DRG-A', 'DRG-B', 'DRG-C'], size=n_patients, p=[0.5, 0.3, 0.2])

df = pd.DataFrame({
    'Patient_ID': [f'P{i:04d}' for i in range(1, n_patients + 1)],
    'DRG_Group': drg_groups,
    'Total_Cost': base_costs
})

# 故意植入几个极端的超高费用病例（异常值）
outlier_indices = np.random.choice(df.index, size=15, replace=False)
df.loc[outlier_indices, 'Total_Cost'] = df.loc[outlier_indices, 'Total_Cost'] * np.random.uniform(10, 100)

# 2. 计算 DRG-A 的 IQR 及异常值边界
df_drga = df[df['DRG_Group'] == 'DRG-A'].copy()
Q1 = df_drga['Total_Cost'].quantile(0.25)
Q3 = df_drga['Total_Cost'].quantile(0.75)
IQR = Q3 - Q1
upper_bound = Q3 + 1.5 * IQR

print(f"DRG-A 费用上界阈值: {upper_bound:.2f} 元")
outliers = df_drga[df_drga['Total_Cost'] > upper_bound]
print(f"共发现 {len(outliers)} 例极端超支病例。")

# 3. 数据可视化：箱线图结合抖动散点图
plt.figure(figsize=(10, 6))
sns.set_theme(style="whitegrid")

# 绘制箱线图，不显示默认的异常值点（由stripplot叠加显示）
ax = sns.boxplot(x='DRG_Group', y='Total_Cost', data=df,
                 showfliers=False, width=0.5, palette="Set2")
```

```
# 叠加抖动散点图以展示每一个病人的实际分布
sns.stripplot(x='DRG_Group', y='Total_Cost', data=df,
              size=4, color=".3", alpha=0.6, jitter=True)

# 标注 DRG-A 的异常值阈值线
ax.axhline(upper_bound, color='red', ls='--', lw=2, label=f'DRG-A Outlier Threshold ({upper_bou

plt.title('DRG Groups Total Cost Distribution with Extreme Cases Identified', fontsize=14, font
plt.ylabel('Total Cost (CNY)', fontsize=12)
plt.xlabel('DRG Group', fontsize=12)
plt.legend()
plt.tight_layout()
plt.show()
```

## 3.2 关系探索与多维比对：从热力图到森林图的循证医学视角

单变量的分布和异常识别只是基础。在复杂的医疗环境中，我们需要探究变量间的相互影响（如费用与住院天数、药占比与治愈率），以及在不同分类（如不同医生、不同病区）下的绩效差异。

### 3.2.1 相关性分析热力图（Correlation Heatmap）

相关矩阵热力图用于快速识别高维数据中的强相关变量对。

- **Pearson 相关系数 ( $r$ )**：衡量连续变量间的线性相关性。取值范围  $[-1, 1]$ 。
- **Spearman 秩相关系数 ( $\rho$ )**：非参数检验，对非正态分布和非线性单调关系更稳健，尤其适用于含有极端值费用的医疗数据。在病种分析中，常用于分析各项明细费用（药品费、耗材费、检验费、手术费）与总费用的关联强度，从而锁定成本管控的重点靶位。

### 3.2.2 森林图（Forest Plot）：多维度绩效差异的量化利器

**森林图理论深度解析：**森林图最初在循证医学的荟萃分析（Meta-analysis）中广泛应用，现已成为医院精细化管理中比较不同主体（医生、科室）绩效的强大工具。相较于简单的柱状图，森林图不仅展示了点估计值（如平均费用差异），还清晰地表达了估计的不确定性（置信区间 Confidence Intervals, CIs）。

- **效应量 (Effect Size)**：在此场景下，可以定义为某位医生的某病种平均费用与科室整体（或标准）平均费用之间的差值（Mean Difference）。
- **95% 置信区间 (95% CI)**：基于样本量和方差计算出的一个区间，表示有 95% 的概率真实总体差值落在此区间内。如果某位医生的 95% CI 跨越了 0（垂直无效线），则在统计学上

不能认为其费用与总体有显著差异 ( $p \geq 0.05$ )。

- **P值 (p-value)**: 拒绝原假设 (两者无差异) 的概率阈值。

### 3.2.3 Python 实战：绘制五位医生单病种平均费用差异森林图

本例通过模拟同一病种（如腹腔镜胆囊切除术）下 5 位主刀医生的费用数据，利用 `scipy.stats` 计算独立样本 t 检验的均值差及其置信区间，并使用纯 `matplotlib` 绘制极具专业学术质感的森林图。

```
import numpy as np
import pandas as pd
import scipy.stats as stats
import matplotlib.pyplot as plt

# 1. 模拟数据：同一病种，5位医生的患者住院费用
np.random.seed(101)
doctors = ['Dr. Smith', 'Dr. Jones', 'Dr. Wang', 'Dr. Patel', 'Dr. Garcia']
# 设定全体基准均值为 15000 元，标准差 2000
global_mean = 15000
global_std = 2000

data = []
# 赋予不同医生不同的均值和病例数
params = {
    'Dr. Smith': {'n': 45, 'mean_offset': -1200, 'std': 1800}, # 控费较好
    'Dr. Jones': {'n': 30, 'mean_offset': +300, 'std': 2200}, # 与均值相近
    'Dr. Wang': {'n': 60, 'mean_offset': +2500, 'std': 2500}, # 费用偏高显著
    'Dr. Patel': {'n': 25, 'mean_offset': -300, 'std': 1900}, # 与均值相近
    'Dr. Garcia': {'n': 40, 'mean_offset': -2100, 'std': 1500} # 控费极好
}

for doc in doctors:
    p = params[doc]
    costs = np.random.normal(loc=global_mean + p['mean_offset'], scale=p['std'], size=p['n'])
    data.extend([(doc, c) for c in costs])

df = pd.DataFrame(data, columns=['Doctor', 'Cost'])
global_population = df['Cost'].values

# 2. 统计计算：计算每位医生与总体平均水平的差值及 95% CI
results = []
for doc in doctors:
    doc_costs = df[df['Doctor'] == doc]['Cost'].values
    n = len(doc_costs)
```

```

# 均值差
mean_diff = np.mean(doc_costs) - np.mean(global_population)

# 执行两独立样本 t 检验 (假设方差不等 Welch's t-test)
t_stat, p_val = stats.ttest_ind(doc_costs, global_population, equal_var=False)

# 计算均值差的标准误 (Standard Error of the Difference)
se_doc = np.std(doc_costs, ddof=1) / np.sqrt(n)
se_glob = np.std(global_population, ddof=1) / np.sqrt(len(global_population))
se_diff = np.sqrt(se_doc**2 + se_glob**2)

# 自由度近似 (Welch-Satterthwaite equation)
df_welch = (se_doc**2 + se_glob**2)**2 / ((se_doc**2)**2/(n-1) + (se_glob**2)**2/(len(global_population)-1))

# 95% 置信区间
t_crit = stats.t.ppf(0.975, df_welch)
ci_lower = mean_diff - t_crit * se_diff
ci_upper = mean_diff + t_crit * se_diff

results.append({
    'Doctor': doc, 'N': n, 'Mean_Diff': mean_diff,
    'CI_Lower': ci_lower, 'CI_Upper': ci_upper, 'p_value': p_val
})

res_df = pd.DataFrame(results).sort_values(by='Mean_Diff', ascending=False).reset_index(drop=True)

# 3. 绘制森林图 (Forest Plot)
fig, ax = plt.subplots(figsize=(9, 6))

y_ticks = np.arange(len(res_df))
ax.set_yticks(y_ticks)
ax.set_yticklabels(res_df['Doctor'])

# 绘制中心垂直无效线 (无效线代表差异为0)
ax.axvline(x=0, color='gray', linestyle='--', linewidth=1.5)

# 绘制每一行的误差线 (CI) 和点估计值
for i, row in res_df.iterrows():
    # 颜色区分: 显著高于0(红), 显著低于0(绿), 不显著(灰)
    if row['CI_Lower'] > 0:
        color = '#d62728' # Red
    elif row['CI_Upper'] < 0:
        color = '#2ca02c' # Green
    else:
        color = '#7f7f7f' # Gray

    ax.plot([row['CI_Lower'], row['CI_Upper']], [i, i], color=color, linewidth=2, marker='|', n

```

```

ax.plot(row['Mean_Diff'], i, marker='s', color=color, markersize=8)

# 在右侧标注具体数值和 P 值
p_text = f"p < 0.001" if row['p_value'] < 0.001 else f"p = {row['p_value']:.3f}"
text_label = f"{row['Mean_Diff']:+.0f} [{row['CI_Lower']:+.0f}, {row['CI_Upper']:+.0f}]\n{p_text}"
ax.text(np.max(res_df['CI_Upper']) + 500, i, text_label, va='center', fontsize=9)

ax.invert_yaxis() # 让排名第一的在上方
ax.set_xlabel('Mean Cost Difference from Global Average (CNY)', fontsize=12)
ax.set_title('Forest Plot: Doctor Specific Cost Difference (Single Disease)', fontsize=14, font
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

# 扩展x轴边界以容纳右侧文字
x_min, x_max = ax.get_xlim()
ax.set_xlim(x_min, x_max + 3000)

plt.tight_layout()
plt.show()

```

*洞察与解读：通过这张森林图，不仅能够直观地看到 Dr. Wang 均费高出总体约 2500 元，更关键的是其置信区间的下界远远跨过了 0 线，且  $p < 0.001$ ，这为管理层约谈该医生提供了无可辩驳的统计学铁证。反之，那些置信区间穿过 0 线的医生，其均费的波动可视为随机误差，无需过度干预。*

## 3.3 战略定位与精益监控：从 BCG 矩阵到休哈特控制图

站在科室或全院的主任/院长视角，病种不仅需要微观的“查错”，更需要宏观的“排兵布阵”与动态的质量控制。

### 3.3.1 波士顿矩阵（BCG Matrix）与战略定位

我们将传统的 BCG 矩阵引入医院管理。横轴通常设定为 **CMI (Case Mix Index, 反映技术难度与收治病种的复杂程度)**，纵轴设定为 **盈亏/结余率 (Profit Margin)**，气泡大小代表 **出院人次 (Volume/权重)**。

- **明星病种 (Star)**：高 CMI，高结余。医院的核心竞争力，应全力扩张床位和资源倾斜。
- **现金牛病种 (Cash Cow)**：低 CMI，高结余。常见多发病，标准化程度高，是医院现金流的压舱石，需保持极高的周转率。
- **问题病种 (Question Mark)**：高 CMI，低结余（甚至亏损）。技术难度大但成本控制极差，需组织多学科 MDT 或攻关小组优化临床路径，否则越做越亏。
- **瘦狗病种 (Dog)**：低 CMI，低结余。应当战略性舍弃或转移至日间手术/下级社区医疗机构。

### 3.3.2 帕累托图 (Pareto Chart)

基于“二八法则” (80/20 Rule)，将导致某问题（如并发症发生、医保拒付原因、超时住院）的因素按发生频率降序排列，结合累积百分比折线，快速锁定需要解决的“关键少数” (Vital Few)。

### 3.3.3 控制图 (Control Chart): 动态质量监控

在六西格玛 (Six Sigma) 精益管理中，休哈特控制图 (Shewhart Control Chart) 通过设定中心线 (Center Line, CL)、控制上限 ( $UCL = CL + 3\sigma$ ) 和控制下限 ( $LCL = CL - 3\sigma$ )，区分系统内的普通原因变异 (在控制限内随机波动) 和特殊原因变异 (突破控制限或出现连续同侧异常趋势)。

### 3.3.4 Python 实战：绘制基于 Plotly 的动态波士顿气泡矩阵与并发症监控控制图

波士顿矩阵气泡图代码 (交互式 Plotly):

```
import plotly.express as px
import pandas as pd
import numpy as np

# 1. 模拟科室各 DRG 病种年度数据
np.random.seed(42)
drgs = [f'DRG-{i:02d}' for i in range(1, 21)]
cmis = np.random.uniform(0.5, 3.5, 20)
profits = np.random.normal(loc=0, scale=5000, size=20) # 有盈有亏
volumes = np.random.randint(50, 1000, 20)

df_bcg = pd.DataFrame({'DRG': drgs, 'CMI': cmis, 'Profit': profits, 'Volume': volumes})

# 计算象限划分的基准线 (通常取中位数)
cmi_mid = df_bcg['CMI'].median()
profit_mid = df_bcg['Profit'].median()

# 2. 绘制 Plotly 气泡图
fig = px.scatter(df_bcg, x="CMI", y="Profit", size="Volume", color="Profit",
                 hover_name="DRG", size_max=40, color_continuous_scale=px.colors.diverging.RdYl,
                 title="DRG Strategic Positioning (Boston Matrix)")

# 添加象限分割线
fig.add_hline(y=profit_mid, line_dash="dash", line_color="gray")
fig.add_vline(x=cmi_mid, line_dash="dash", line_color="gray")

# 添加象限注释
```

```

fig.add_annotation(x=3.2, y=max(profits)-1000, text="Star (明星)", showarrow=False, font=dict(s
fig.add_annotation(x=0.8, y=max(profits)-1000, text="Cash Cow (现金牛)", showarrow=False, font=
fig.add_annotation(x=3.2, y=min(profits)+1000, text="Question Mark (问题)", showarrow=False, fc
fig.add_annotation(x=0.8, y=min(profits)+1000, text="Dog (瘦狗)", showarrow=False, font=dict(si

fig.update_layout(xaxis_title="CMI (Technical Complexity)", yaxis_title="Profit/Loss (CNY)")
# fig.show() # 在 Jupyter 或网页端展示交互图

```

## 控制图 (Control Chart) 代码： 监测长达 24 个月的某手术切口感染率。

```

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# 1. 模拟数据：24个月的感染率（百分比）
months = pd.date_range(start='2022-01-01', periods=24, freq='M')
np.random.seed(99)
infection_rates = np.random.normal(loc=2.5, scale=0.5, size=24)
# 在第 18、19 个月制造特殊原因变异（异常飙升）
infection_rates[17] = 4.8
infection_rates[18] = 4.2

df_cc = pd.DataFrame({'Date': months, 'Rate': infection_rates})

# 2. 计算控制线
mean_rate = df_cc['Rate'].mean()
std_rate = df_cc['Rate'].std()
ucl = mean_rate + 3 * std_rate
lcl = max(0, mean_rate - 3 * std_rate) # 感染率不可能小于0

# 3. 绘制 X-bar 控制图
plt.figure(figsize=(12, 5))
plt.plot(df_cc['Date'], df_cc['Rate'], marker='o', linestyle='-', color='#1f77b4', label='Month
plt.axhline(mean_rate, color='green', linestyle='-', linewidth=2, label=f'Center Line (Mean): {
plt.axhline(ucl, color='red', linestyle='--', linewidth=2, label=f'UCL (+3σ): {ucl:.2f}%')
plt.axhline(lcl, color='red', linestyle='--', linewidth=2, label=f'LCL (-3σ): {lcl:.2f}%')

# 标记越界点
out_of_control = df_cc[df_cc['Rate'] > ucl]
if not out_of_control.empty:
    plt.scatter(out_of_control['Date'], out_of_control['Rate'], color='red', s=100, zorder=5, l

plt.title('Statistical Process Control (SPC) Chart: Post-op Infection Rate', fontsize=14, fontw
plt.ylabel('Infection Rate (%)', fontsize=12)
plt.xticks(rotation=45)

```



```
plt.legend(loc='upper left')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

## 3.4 进阶：归因与预测模型 —— 开启机器学习的黑盒 (SHAP 分析)

在模块二中，我们探讨了应用多元线性回归或逻辑回归等传统统计模型去寻找影响因素，以及应用决策树、随机森林、XGBoost 等机器学习模型去预测费用超支或高危再入院。

传统回归模型的优势在于其天然的解释性（回归系数  $\beta$ ），但对非线性关系和多重共线性处理较弱；而 XGBoost 等树基集成模型预测精准，但一直被诟病为“黑盒”（Black Box），难以告诉临床医生“究竟为什么这个特定病人会亏损这么多？”

### 3.4.1 引入 SHAP (Shapley Additive exPlanations)

为了兼顾预测高精度与临床可解释性，我们引入博弈论中的 Shapley 值概念。SHAP 是一个统一的解释框架，它将模型预测的输出值，精确地分配（归因）给每一个输入的特征变量。

- **全局解释 (Global Interpretability)**：整体来看，哪些特征最重要？正向还是负向影响？
- **局部解释 (Local Interpretability)**：针对当前这一个亏损 5 万块钱的特大超支病例，究竟是“ICU 停留时间长”贡献了 3 万，还是“特定昂贵耗材”贡献了 2.5 万，亦或者是“患者年龄较轻”抵消了 5 千块钱的成本？

### 3.4.2 Python 实战：基于 XGBoost 与 SHAP 解码单个超支病例的亏损归因

以下代码演示了如何训练一个简单的基于树的回归模型来预测患者费用超支金额，并使用 shap 库生成极为直观的瀑布图（Waterfall Plot），解析单一患者的具体归因路径。

```
import numpy as np
import pandas as pd
import xgboost as xgb
import shap

# 1. 模拟数据集：预测患者的超支金额 (Loss_Amount)
np.random.seed(123)
n = 1000
age = np.random.randint(18, 90, n)
los = np.random.normal(7, 3, n) # Length of stay
```

```

icu_days = np.random.choice([0, 1, 2, 5, 10], size=n, p=[0.7, 0.1, 0.1, 0.05, 0.05])
comorbidity_idx = np.random.randint(0, 5, n)
surgery_complexity = np.random.randint(1, 4, n)

# 构造一个非线性的目标变量：超支金额
base_loss = -5000 + (age * 50) + (los * 800) + (icu_days * 3500) + (comorbidity_idx * 1200) + (
# 添加随机噪声
loss_amount = base_loss + np.random.normal(0, 2000, n)

X = pd.DataFrame({
    'Age': age,
    'LOS_Days': los,
    'ICU_Days': icu_days,
    'Comorbidity_Index': comorbidity_idx,
    'Surgery_Complexity': surgery_complexity
})
y = loss_amount

# 2. 训练 XGBoost 树模型
model = xgb.XGBRegressor(n_estimators=100, max_depth=4, learning_rate=0.1, random_state=42)
model.fit(X, y)

# 3. 初始化 SHAP 解释器
# TreeExplainer 专为 XGBoost/LightGBM 等树模型高度优化
explainer = shap.TreeExplainer(model)
shap_values = explainer(X)

# 4. 局部解释 (Local Interpretability): 剖析全场亏损最严重的“倒霉蛋”病例
# 找到亏损额最大的患者索引
worst_patient_idx = np.argmax(y)
worst_loss = y[worst_patient_idx]
print(f"最严重超支病例 Index: {worst_patient_idx}, 实际超支预测值: {model.predict(X.iloc[[worst_patient_idx])}]")
print(f"该病例特征特征详情:\n{X.iloc[worst_patient_idx]}")

# 绘制瀑布图 (Waterfall Plot) 解释该特定患者的超支路径
# 注意: 需要在支持 matplotlib 输出的环境 (如 Jupyter Notebook) 中运行
shap.plots.waterfall(shap_values[worst_patient_idx], max_display=10)

```

**瀑布图原理解读：** - 瀑布图的底部基线 ( $E[f(x)]$ ) 代表了全样本的平均超支预测值。 - 图的顶部 ( $f(x)$ ) 代表了**该特定患者**的最终预测超支金额。 - 中间的条形代表了各个特征 (如 `ICU_Days = 10`) 是如何一步步将预测值从总体均值“推高”或“拉低”到最终结果的。红色条形代表正向增加亏损 (例如长期的 ICU 滞留), 蓝色条形代表减少亏损的因素。 - 这种高度定制化的个体归因图, 可以直接作为医务处查房和病案反馈的终极武器, 让主治医生对超支的原因心服口服。

(模块三结语：可视化和算法不是为了炫技，而是为了在茫茫数据中定位管理焦点。从箱线图的基线管控，到森林图的循证比较，再到 SHAP 的微观剖析，我们的最终目的是重塑医院精益运营的管理闭环。)